EGG CATCHER (9-13) #Individual #Game_creation #Random_numbers

Part 0. Before you begin

Required materials:

- a sample solution file to be demonstrated to students
- A4 sheets (one sheet per student)
- Optionally: Printed cards to demonstrate game rules.

Teaching materials:

❏ Link to the type 1 sample solution.
❏ Link to the type 2 sample solution.

Recommended implementation time for the project: Two lessons.

Part 1. General project information

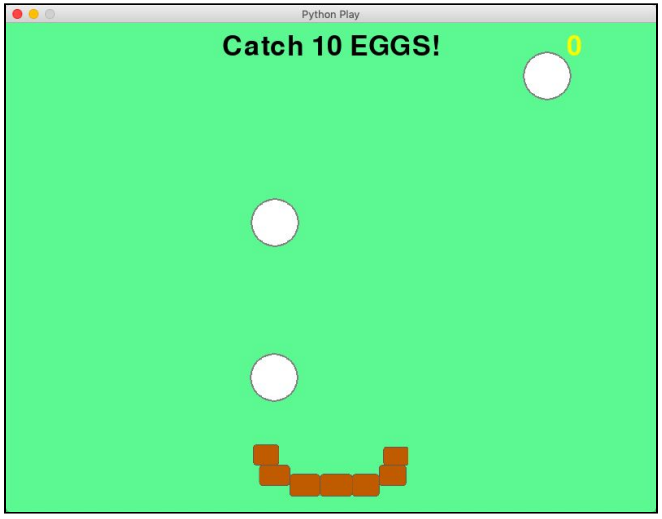
Minimal prior knowledge required from students		*Learning results*
Each student: - is familiar with **random** module and is capable of working with randint() - can create **lists** and use append() and remove() methods knowingly - can use pygame to load **background music** - can use **async...await** construct to pause the game - handles **keyboard events** confidently using play		- Student learned what **time module** is and how to use it to get system time and game event settings - Student learned/reviewed the concept of **global** variables and used them in his/her program. - Student **created an arcade game** using his/her entire obtained knowledge *Ideas*: - A student became interested in thinking through computer activity against user and wants to create a more difficult game (make computer "even smarter").

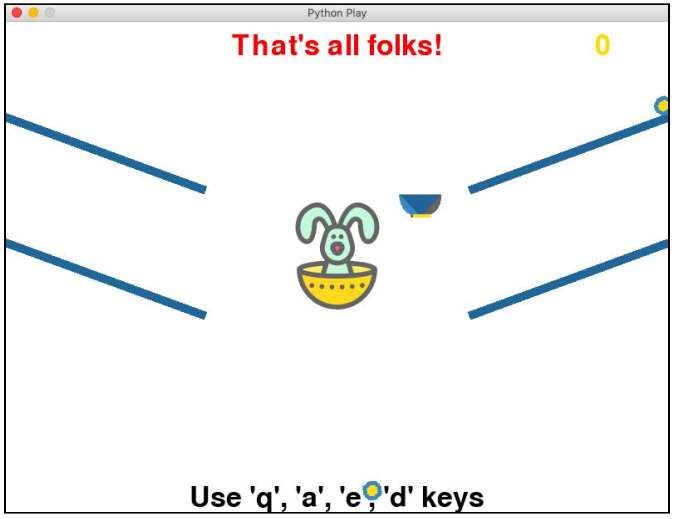
Part 2. Description of the expected results

Program	*Features*
Egg catcher	• When program starts, it welcomes user and offers to catch 10 eggs. Caught eggs counter (initially 0) is displayed on the side. Also displayed are egg basket and an egg which immediately starts dropping on the floor. • Each 3 seconds an egg appears at the top and starts to drop. If an egg touches the basket, it is considered to be caught. If an egg does not touch the basket and gets below it, the egg "breaks" and user loses. • User wins if he/she catches 10 eggs, dropping none on the floor.

Part 3. Project stages

IMPORTANT NOTICE. The feature of "Egg breaker" is that student decides on game's work principles.
First version of "Egg catcher": eggs appear in random positions and fall down from above. Second version (difficult): eggs slide down the boards.





Student who chooses Egg Catcher game also needs to select its variant. Basket variant is easier and well-suited for students who have completed Arkanoid. Rabbit variant is more difficult and is suited for advanced students.

No.	Stages and their goals	Content	Format	Results
1	**"Projects fair"** Select from the list of available projects Get a functional specification (FS). Review the requirements and take on the project	Each student can select a project to work on. Then go to the platform and review functional specifications for the projects (**"Blackjack: FS" or "Egg catcher: FS"**). **Note**. Notice that "Egg Catcher" has two possible implementations, and you need to select one of them in FS. After the review, students should get started with the first task — create a project checklist.	Work with a teacher.	Each student has picked up a project. Each student has received and reviewed an FS, and is ready to get started.

The description provided below only relates to Egg Catcher project stages.

2	**Creating a project check list** Sign in to the platform. Locate the second level of **"Egg Catcher: FS"** (check list requirements are provided at the second level) Assess whether familiar tools and theory are sufficient to complete the project (yes) Create a project check list on an A4 sheet on your own or with teacher's support	Arrange check list creation. **Student's purpose is to describe sprite programming and game event handling sequence.** Recommend students to open "Egg Catcher: FS" assignment and locate the slide containing project check list requirements. If a student is at a loss, start from launching the finished project and help him/her to name all sprites (basket, eggs, counter, hint sprite) and events of the game (egg and basket contact, egg and ground contact, keyboard events).	Individual work, work with a teacher	Student has created the project check list. **A possible project mind map is provided in part 4 "Sample project implementation".*

		Keep in mind that the more difficult programming is for the student, the more detailed the project check list needs to be (**a sample check list is provided at the end of methodological guidelines**).		

3	**Adding sprites.** **Program first startup.** Open the project draft (egg1.py or egg2.py file as per selected variant of Egg Catcher) Add egg sprites. Launch the program and eliminate any errors.	Arrange individual work. Offer student to open VSC, enter his/her login and password for mars.algoritmika.org and open "Egg Catcher: VSC assignment. Student must start implementing the first part of the program: making a user's move. Point out to the student that he/she starts working in egg1.py or egg2.py file (as per selected Egg Catcher variant) containing the project draft. **"Egg Catcher", variant 1**. We need to add egg sprites. Create an egg: `egg=play.new_circle(color='white', x=0, y=0, radius=30, border_color='grey', border_width=2)` Because there will be many eggs, and they will have to be destroyed once they contact with the basket, we shall store them in a list. Yet, put the newly created egg in there: `eggs=[egg]` Let it appear in a random location at the top of the window. Y coordinate has to be tailored to window height. X coordinate is set randomly with randint() function using window width:	Individual work, work in pairs. * Creating sprites should be fairly easy, so stick to individual work as much as possible.	Egg Catcher with all required sprites. Sprites are still inactive.

		`` @play.when_program_starts def start(): eggs[0].x = randint(-play.screen.width/2+20, play.screen.width/2-20) eggs[0].y = play.screen.height/2-20 ``` **Egg Catcher, variant 2.** Write new_egg() function to create a new egg and add it to the egg list. Actions are similar to those in Egg Catcher variant 1, however, we suggest to put them in a separate function. See a sample solution at the end of methodological guidelines.		

The necessary theory for the implementation of the next stage should be known to the student, but help of a teacher is required for its application.

4	**Basket movement** Think up how egg basket will move. Program basket movement with keypresses.	Arrange individual work. Keyboard events have already been handled before, so a student has to try to do the task on his/her own. If the student finds it hard to do the task, remind him/her of core play.key_is_pressed() method and suggest to look for a solution in "Maze" project. **"Egg Catcher", variant 1**. Sprite can be moved using physical object speed (smooth movement) or sprite coordinates (easier, but movement will be more abrupt). We suggest to implement the basket as a physical object, and egg drop using coordinates. Enable physics in @play.when_program_starts section: ``` backet.start_physics(stable=True, obeys_gravity=False, bounciness=1, mass=10) ```	Individual work.	

		Basket movement is described in @play.repeat_forever section: `# an example of movement on A key press` `if play.key_is_pressed('a'):` `backet.physics.x_speed = -15` **Egg Catcher, variant 2.** In this case, basket will not move along the screen bottom but immediately arrive at one of four points (next to one of the boards where the egg slides down). Sprite movement to a certain point (which needs to be determined) when one of keys is pressed has to be programmed. Student can do this stage on his/her own. Movement can be programmed as follows: `@play.repeat_forever` `async def game():` `# control put the basket in one of four points` `if play.key_is_pressed('e') or play.key_is_pressed('y'):` `bowl.x = 100` `bowl.y = 80`		
5	**Programming of egg appearing and dropping** Think up how eggs will appear each 3 seconds in	**"Egg Catcher", variant 1**. Eggs must appear each 3 seconds which need to be read from the computer's system time. Use time module to do this. Tell the students to open a hint with time module commands in the assignment and explore it. Then a student has to try to program	Individual work, work in pairs, work with a teacher. **Help a student to*	All events are handled and sprites are created in the program.

	different spots. Program egg dropping.	the action execution each 3 seconds. Create a global variable `old_time = 0`. At program startup, assign current system time to the variable: `def start():` `  global old_time` `  old_time = time.time()` Then address this variable in game(). To perform further actions each 3 seconds, difference between current and previous time must exceed 3 seconds: `  if time.time()-old_time > 3:` `  #here a new egg will be created` `  old_time = time.time()` A new egg is created similarly to the first one and added to the list. Student can do this step on his/her own. Student must also try to program egg dropping and contact between egg and basket without assistance. If these attempts fail, arrange pairwork. Direct the pairwork with following questions: "How do I move a sprite using coordinates? What do I do to move a sprite down by constant distance? Which method determines if sprites have come in contact?" Notice that moving sprites and checking if they have contacted the basket is done continuously for all list elements. `  #egg  dropping and contact with basket`	*use global variables and time module.*	The program can not yet determine loser and winner.

```
    for egg in eggs:
        if egg.is_touching(backet):
            eggs.remove(egg)
            egg.hide()
            eggs_amount.words=str(int(eggs_amount.words)+1)
        egg.y=egg.y-5
```

Egg Catcher, variant 2.

Egg dropping and contact with basket is programmed in a way similar to the first variant of Egg Catcher. Program must continuously check for each egg if it has run into the basket. If it has, egg is considered to be caught and is deleted from eggs list.

```
        for egg in eggs:
            if egg.is_touching(bowl):
                egg.hide()
                eggs.remove(egg)
                score = score + 1
                score_txt.words = score
```

New eggs are created each 3 seconds similarly to Egg Catcher variant 1, but using new_egg() function:

```
    #new eggs
    if time.time()-old_time > 3:
        new_egg()
        old_time = time.time()
```

6	**Win and lose situations**	Stage is set for handling win and lose situations. These cases are described in @play.repeat_forever. Offer students to program this stage without assistance. **Note**. Advise students to clearly show to the user that the game is over. To do this, call quit() function at the end of a won/lost game; application window will be closed. **"Egg Catcher", variant 1**. Victory example: ❏ User wins if he/she has caught 10 eggs and none of them was broken. `#victory` `if int(eggs_amount.words) == 10:` `lose = play.new_text(words='YOU WIN', x=0, y=0, color='yellow', font_size=100)` `backet.hide()` `await play.timer(seconds=1)` `quit()` Defeat is programmed in a similar way. See the complete possible solution below. **Egg Catcher, variant 2**. Defeat example: ❏ Defeat should be included in the loop checking contact between egg and basket. If an egg ends up below the	Individual work, work in pairs. *At this stage, no pair debate is intended. However, do not prevent one student from helping another.	The program is complete. You need to test it and deliver the project to customer.

		basket, it is considered as broken, and the game ends. `for egg in eggs:` `  #losing` `  if egg.y < -250:` `  hello_txt.words = "That's all folks!"` `  hello_txt.color = 'red'` `  quit()` `  if egg.is_touching(bowl):` `  #...`		
7	**Project delivery (testing by teacher)**	Once a student declares project completion, arrange testing. It is performed in three stages: - Testing by project author. Student must open the project description and check if required actions are performed correctly. - Mutual project testing by students. Performed similarly to code review. - Project testing by teacher. Check the program. If everything works correctly, ask student comprehension questions on the topic. If student answers those successfully, offer additional tasks.	Individual work, work in pairs, work with a teacher.	Basic part of the project is complete. If there is enough time, student starts doing additional tasks.

🎯 Part 4. Sample project implementation

Project checklist:

- Add egg sprites
- Program basket movement with keyboard presses

- Program egg dropping
- Program egg contacting basket (incremented the caught egg counter)
- Program visual design of victory and defeat.

Sample solution:

- Egg Catcher, variant 1.
- Egg Catcher, variant 2.

🎯 Part 5. Result evaluation methods

Stage 1. *Program operability check*

Any program must handle the following test cases. If the test is passed, student is considered to achieve mandatory comprehension level.

Program testing	
• Program is started, nothing pressed	• New eggs are generated each 3 seconds and drop down
• One of the basket control keys is pressed	• Basket is moved in accordance with key purpose (e.g. A key moves the basket to the left)
• Egg touches the basket	• Egg is considered to be caught (it disappears), egg counter is incremented.
Program notifies user once an egg has dropped onto the floor (defeat) or 10 eggs have been caught (victory)	

Stage 2. *Student knowledge level inspection.*

Note. Teacher uses this section not to evaluate student publicly but personally assess his/her level of knowledge in the group.

There were no new advanced solutions in "Egg Catcher" project. Rather, this program summarizes knowledge gained in the project module. This is why a detailed knowledge check is not required at this stage. The best material comprehension indicator will be solving one of additional problems.

🎯 Part 6. Additional tasks for project improvement

Students wishing to improve their project can be offered additional tasks.

Task 1. Set the background music! Download a music file from the Internet and add it to the project in "Add file" tab. Then load Pygame, add music to the program and enable it in @play.when_program_start section. Or use hello.mp3 file.

Answer for this task, see in Egg Catcher variant 1.

Task 2. Make the game speed gradually increase over time. Which variables do you need to do this, and how will they impact gameplay?

Answer for this task, see in Egg Catcher variant 2.

EGG CATCHER

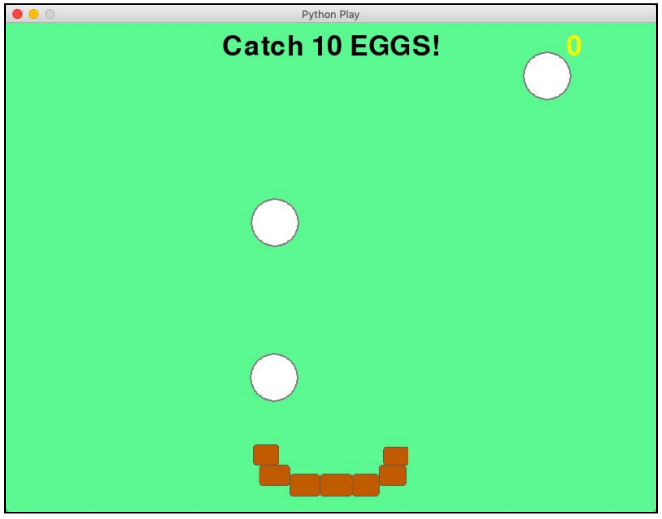
#Individual #Game_creation #Random_numbers

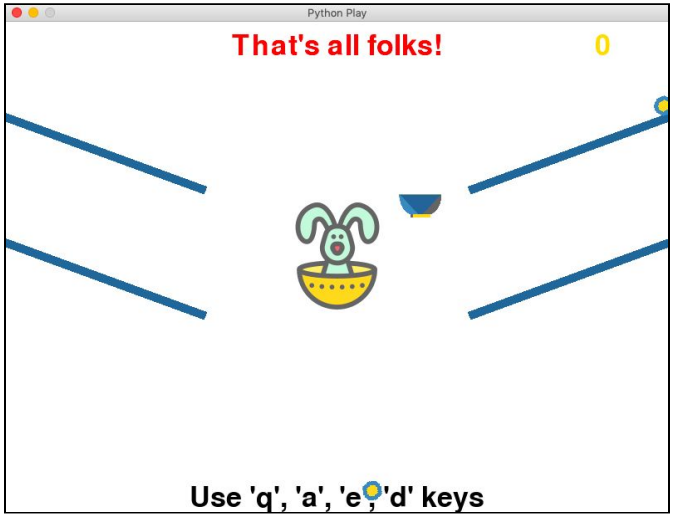
Dear developer, you have been assigned an order for **EGG CATCHER** program.

PROJECT:

- When program starts, it welcomes user and offers to catch 10 eggs. Caught eggs counter (initially 0) is displayed on the side. Also displayed are egg basket and an egg which immediately starts dropping on the floor.
- Each 3 seconds an egg appears at the top and starts to drop. If an egg touches the basket, it is considered to be caught. If an egg does not touch the basket and gets below it, the egg "breaks" and user loses.

You can select one of two game variants:





User loses if at least one egg has dropped on the floor. **User wins if** he/she has collected 10 eggs.

It is recommended to implement this program in Python **over 2 lessons.**
Testing is a critical software development stage. It consists of 3 stages:

- *Independent program operability check*. For example, a check if victory and defeat conditions work must be performed.
- *Program peer review by a developer*.

After successful testing, the project is ready for **presenting to the customer**. This is the final stage of testing the project.